

SUPERSET ID - 6364957 (Sachin Ray)

Complete Git Hands-On Lab Solutions

Lab 1: Git Setup and Configuration

Objective: Learn basic Git commands and configuration.

Step 1: Setting up Git Configuration

1.1 Verifying Git Installation

```
git --version
```

Output:

```
KIIT@SACHIN MINGW64 ~ (main)
$ git --version
git version 2.47.0.windows.1
```

1.2 Configuring User Information

```
git config --global user.name "Sachin"
git config --global user.email "sachin122np@gmail.com"
```

1.3 Verifying Configuration

```
git config --global --list
```

Output:

```
KIIT@SACHIN MINGW64 ~ (main)
$ git config --global user.name "Sachin"

KIIT@SACHIN MINGW64 ~ (main)
$ git config --global user.email "sachin122np@gmail.com"
```

```
KIIT@SACHIN MINGW64 ~ (main)
$ git config --global --list
user.name=Sachin
user.email=sachin122np@gmail.com
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
```

Step 2: Configuring Notepad++ as Default Editor

2.1 Testing Notepad++ Command

```
notepad++
```

```
KIIT@SACHIN MINGW64 ~ (main)
$ notepad++
bash: notepad++: command not found
```

2.2 Creating Alias for Notepad++

```
alias npp='notepad++.exe'
```

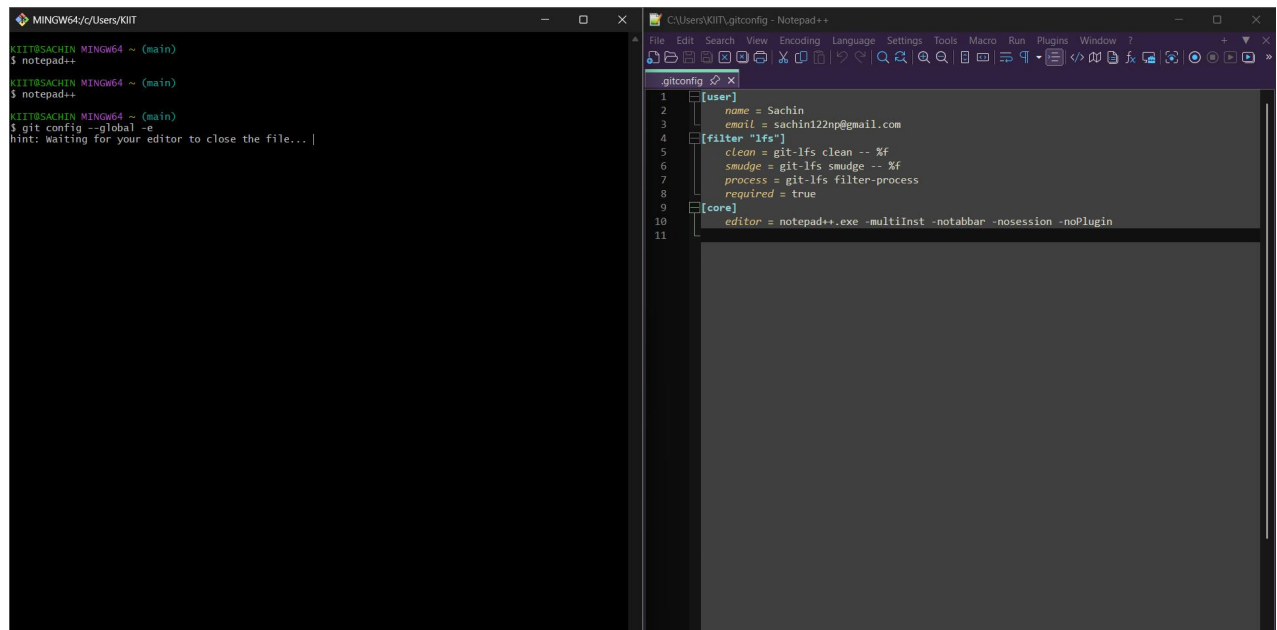
2.3 Setting Notepad++ as Git Editor

```
git config --global core.editor "notepad++.exe -multiInst -notabbar -nosession -noPlugin"
```

2.4 Testing Editor Configuration

```
git config --global -e
```

Result:



The image shows a terminal window on the left and a Notepad++ editor on the right. The terminal window displays the following commands and output:

```
MINGW64/c/Users/KIIT
KIIT@SACHIN MINGW64 ~ (main)
$ notepad++
KIIT@SACHIN MINGW64 ~ (main)
$ notepad++
KIIT@SACHIN MINGW64 ~ (main)
$ git config --global e
hint: Waiting for your editor to close the file... |
```

The Notepad++ editor shows the contents of the `gitconfig` file:

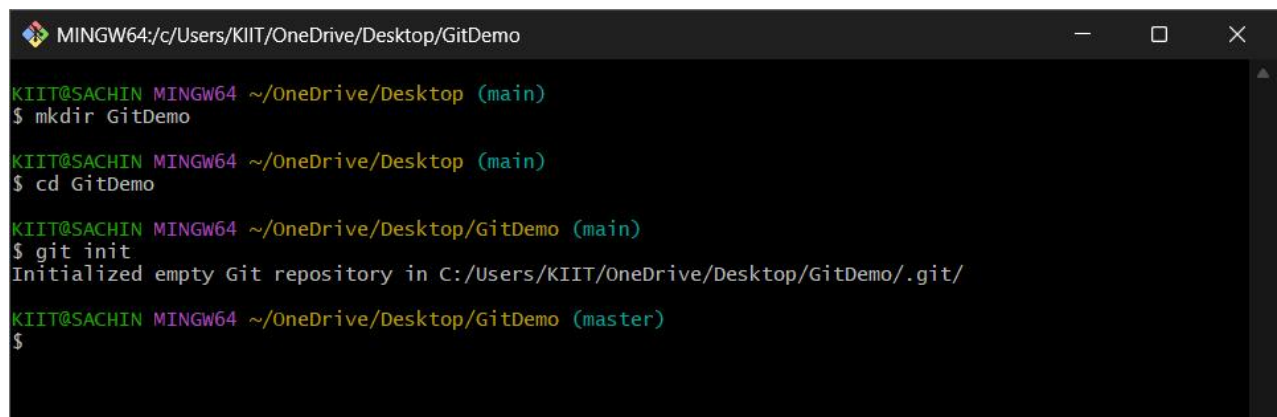
```
1 [user]
2   name = Sachin
3   email = sachin122np@gmail.com
4 [filter "lfs"]
5   clean = git-lfs clean -- %f
6   smudge = git-lfs smudge -- %f
7   process = git-lfs filter-process
8   required = true
9 [core]
10 editor = notepad++.exe -multiInst -notabbar -nosession -noPlugin
11
```

Step 3: Creating and Managing a Repository

3.1 Initializing Repository

```
mkdir GitDemo
cd GitDemo
git init
```

Output:



The image shows a terminal window with the following commands and output:

```
MINGW64/c/Users/KIIT/OneDrive/Desktop/GitDemo
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop (main)
$ mkdir GitDemo
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop (main)
$ cd GitDemo
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (main)
$ git init
Initialized empty Git repository in C:/Users/KIIT/OneDrive/Desktop/GitDemo/.git/
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$
```

3.2 Verifying Repository Structure

```
ls -la
```

Expected Output includes:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ ls -la
total 20
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:04 ./
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:03 ../
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:04 .git/
```

3.3 Creating and Adding Content to File

```
echo "Welcome to Git Demo" > welcome.txt
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ echo "Welcome to Git Demo" > welcome.txt
```

3.4 Verifying File Creation

```
ls -la
cat welcome.txt
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ ls -la
total 21
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:05 ./
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:03 ../
drwxr-xr-x 1 KIIT 197121 0 Jul 30 13:04 .git/
-rw-r--r-- 1 KIIT 197121 20 Jul 30 13:05 welcome.txt

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ cat welcome.txt
Welcome to Git Demo
```

3.5 Checking Git Status

```
git status
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        welcome.txt

nothing added to commit but untracked files present (use "git add" to track)
```

3.6 Adding File to Staging Area

```
git add welcome.txt
```

3.7 Committing Changes

```
git commit -m "Initial commit: Added welcome.txt"
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git add welcome.txt

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git commit -m "Initial commit: Added welcome.txt"
[master (root-commit) c787e55] Initial commit: Added welcome.txt
1 file changed, 1 insertion(+)
create mode 100644 welcome.txt
```

3.8 Verifying Status After Commit

```
git status
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Lab 2: Git Ignore Implementation

Objective: Use .gitignore to exclude files/folders.

2.1 Creating Files to Ignore

```
echo "Debug information" > debug.log
mkdir logs
echo "Application log" > logs/app.log
echo "Error log" > logs/error.log
```

2.2 Checking Status Before .gitignore

```
git status
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ echo "Debug information" > debug.log

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ mkdir logs

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ echo "Application log" > logs/app.log

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ echo "Error log" > logs/error.log

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        debug.log
        logs/

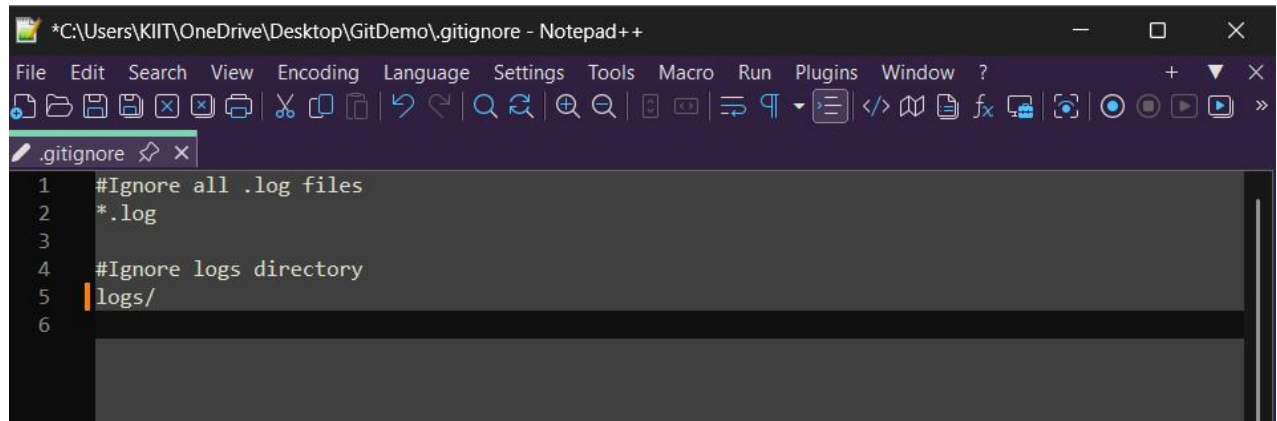
nothing added to commit but untracked files present (use "git add" to track)

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ |
```

2.3 Creating .gitignore File

```
notepad++.exe .gitignore
```

Adding/Editing:



```
*C:\Users\KIIT\OneDrive\Desktop\GitDemo\.gitignore - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
1 #Ignore all .log files
2 *.log
3
4 #Ignore logs directory
5 logs/
6
```

2.4 Verifying .gitignore Effect

```
git status
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore

nothing added to commit but untracked files present (use "git add" to track)
```

2.5 Committing .gitignore

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git commit -m "Added .gitignore to exclude log files and folders"
[master 18f7b37] Added .gitignore to exclude log files and folders
1 file changed, 5 insertions(+)
create mode 100644 .gitignore
```

Lab 3: Branching and Merging

Objective: Practice branch creation, switching, and merging.

3.1 Creating New Branch

```
git branch GitNewBranch
```

3.2 Listing All Branches

```
git branch -a
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git branch GitNewBranch

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git branch -a
  GitNewBranch
* master
```

3.3 Switching to New Branch

```
git checkout GitNewBranch
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'
```

3.4 Adding Files to Branch

```
echo "Feature implementation" > feature.txt
echo "Additional functionality" > addon.txt
```

3.5 Adding and Committing Changes

```
git add .
git commit -m "Added feature files to GitNewBranch"
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitNewBranch)
$ git add .

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitNewBranch)
$ git commit -m "Added feature files to GitNewBranch"
[GitNewBranch 5508f2a] Added feature files to GitNewBranch
 2 files changed, 2 insertions(+)
 create mode 100644 addon.txt
 create mode 100644 feature.txt
```

3.6 Checking Status

```
git status
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean
```

3.7 Switching Back to Master

```
git checkout master
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitNewBranch)
$ git checkout master
Switched to branch 'master'
```

3.8 Comparing Branches

```
git diff master GitNewBranch
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git diff master GitNewBranch
diff --git a/addon.txt b/addon.txt
new file mode 100644
index 0000000..8097100
--- /dev/null
+++ b/addon.txt
@@ -0,0 +1 @@
+Additional functionality
diff --git a/feature.txt b/feature.txt
new file mode 100644
index 0000000..fb6623c
--- /dev/null
+++ b/feature.txt
@@ -0,0 +1 @@
+Feature implementation
```

3.9 Merging Branch to Master

```
git merge GitNewBranch
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git merge GitNewBranch
Updating 18f7b37..5508f2a
Fast-forward
  addon.txt | 1 +
  feature.txt | 1 +
  2 files changed, 2 insertions(+)
  create mode 100644 addon.txt
  create mode 100644 feature.txt
```

3.10 Viewing Merge History

```
git log --oneline --graph --decorate
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git log --oneline --graph --decorate
* 5508f2a (HEAD -> master, GitNewBranch) Added feature files to GitNewBranch
* 18f7b37 Added .gitignore to exclude log files and folders
* c787e55 Initial commit: Added welcome.txt
```

3.11 Deleting Merged Branch

```
git branch -d GitNewBranch
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was 5508f2a).
```

Lab 4: Conflict Resolution

Objective: Learn to handle merge conflicts.

4.1 Verifying Clean Master State

```
git status
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean
```

4.2 Creating and Switching to Work Branch

```
git checkout -b GitWork
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'
```

4.3 Creating hello.xml in Branch

```
echo "<message>Hello from branch</message>" > hello.xml
```

4.4 Committing in Work Branch

```
git add hello.xml
git commit -m "Added hello.xml in GitWork branch"
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitWork)
$ echo "<message>Hello from branch</message>" > hello.xml

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitWork)
$ git commit -m "Added hello.xml in GitWork branch"
[GitWork 0c3b354] Added hello.xml in GitWork branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml
```

4.5 Switching to Master

```
git checkout master
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (GitWork)
$ git checkout master
Switched to branch 'master'
```

4.6 Create Conflicting hello.xml in Master

```
echo "<message>Hello from master</message>" > hello.xml
```

4.7 Committing in Master

```
git add hello.xml
git commit -m "Added hello.xml in master branch"
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ echo "<message>Hello from master</message>" > hello.xml

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git add hello.xml
git commit -m "Added hello.xml in master branch"
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it
[master bbd2324] Added hello.xml in master branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml
```

4.8 Viewing History

```
git log --oneline --graph --decorate --all
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git log --oneline --graph --decorate --all
* bbd2324 (HEAD -> master) Added hello.xml in master branch
| * 0c3b354 (GitWork) Added hello.xml in GitWork branch
|/
* 5508f2a Added feature files to GitNewBranch
* 18f7b37 Added .gitignore to exclude log files and folders
* c787e55 Initial commit: Added welcome.txt
```

4.9 Attempting to Merge

```
git merge GitWork
```

Output:

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git merge GitWork
Auto-merging hello.xml
CONFLICT (add/add): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.
```

4.10 Checking Conflict Status

```
git status
```

```

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both added:      hello.xml

no changes added to commit (use "git add" and/or "git commit -a")

```

4.11 Viewing Conflict Markers

cat hello.xml

```

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master|MERGING)
$ cat hello.xml
<<<<<<< HEAD
<message>Hello from master</message>
=====
<message>Hello from branch</message>
>>>>>>> GitWork

```

4.12 Resolving Conflict Manually

Edit hello.xml:

notepad++.exe hello.xml

--message to edit : <message>Hello from both master and branch</message>

4.13 Adding Resolved File and Commit

git add hello.xml

git commit -m "Resolved merge conflict in hello.xml"

```

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master|MERGING)
$ git commit -m "Resolved merge conflict in hello.xml"
[master 5d08111] Resolved merge conflict in hello.xml

```

4.14 Updating .gitignore for Backup Files

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ notepad++.exe .gitignore
```

Adding to .gitignore:

```
# Backup files
*.orig
```

4.15 Committing .gitignore Update

```
git add .gitignore
git commit -m "Updated .gitignore to exclude backup files"
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git add .gitignore

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git commit -m "Resolved merge conflict in hello.xml"
[master cf321df] Resolved merge conflict in hello.xml
1 file changed, 5 insertions(+), 1 deletion(-)
```

4.16 Deleting Work Branch

```
git branch -d GitWork
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was 0c3b354).
```

4.17 Viewing Final History

```
git log --oneline --graph --decorate
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git log --oneline --graph --decorate
* cf321df (HEAD -> master) Resolved merge conflict in hello.xml
* 5d08111 Resolved merge conflict in hello.xml
| \
| * 0c3b354 Added hello.xml in GitWork branch
* | bbd2324 Added hello.xml in master branch
| /
* 5508f2a Added feature files to GitNewBranch
* 18f7b37 Added .gitignore to exclude log files and folders
* c787e55 Initial commit: Added welcome.txt
```

Lab 5: Remote Repository Operations

Objective: Clean up local repo and push changes to remote.

5.1 Verifying Clean Master State

```
git status
```

```
MINGW64:/c/Users/KIIT/OneDrive/Desktop/GitDemo

KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean
```

5.2 Listing Branches

```
git branch -a
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (master)
$ git branch -a
* master
```

5.3 Adding Remote Repository

```
git remote add origin https://github.com/SACHIN-NP/GitDemo.git
```

5.4 Pulling from Remote

```
git pull origin main
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (main)
$ git pull origin main
From https://github.com/SACHIN-NP/GitDemo
 * branch          main      -> FETCH_HEAD
Already up to date.
```

5.5 Push Changes to Remote

```
git push -u origin main
```

```
KIIT@SACHIN MINGW64 ~/OneDrive/Desktop/GitDemo (main)
$ git push -u origin main
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (22/22), 1.98 KiB | 508.00 KiB/s, done.
Total 22 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/SACHIN-NP/GitDemo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

5.6 Verifying

Online

The screenshot shows the GitHub repository page for 'SACHIN-NP / GitDemo'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows a series of updates to the 'main' branch, including resolving merge conflicts and adding new files. The file list shows the following files: .gitignore, addon.txt, feature.txt, hello.xml, and welcome.txt. The README section is currently empty, with a prompt to 'Add a README'. The right sidebar shows the 'About' section, which is currently empty, and the 'Releases' and 'Packages' sections, which also show no published content.

